Indian Institute of Information Technology, Kota

ECD302 - Skill Transformation based Engineering Project - I

# Hardware-Level Atomic Instruction for Source-Authenticated Sensor Data Acquisition

## Securing RISC-V IoT Endpoints against Software Spoofing

Presented by : Gyan Veer Singh, Chinmay Nirala, Vadekar Baburao, Garapati Mohith

Guided by : Dr. Ashok Kumar Kherodia

# The Core Problem

## IoT Security

- **The Threat :** Data transferred between memory, peripherals, and the CPU is often vulnerable to interception or malicious modification (e.g., bus snooping, fault injection).
- **The Software Flaw :** Traditional systems read sensor data into memory, *then* software adds timestamps, and *then* software encrypts it.
- **The Window of Attack :** Hackers can easily intercept and alter the raw data in the registers or memory between these software execution steps.

# The Solution

## Hardware Based Encryption Technology

**Concept:** Move the security perimeter from software to the hardware data-path itself.

**The Mechanism:** Introduce a custom atomic instruction designed exclusively for secure data acquisition.

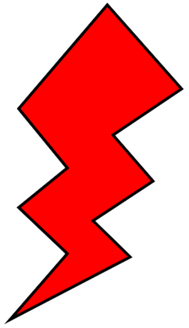
**The Guarantee:** Ensure that when the software receives a data packet, it is mathematically proven to have originated from a specific sensor at a specific time, with zero chance of software interception.

# Industry Parallels and Academic Foundations

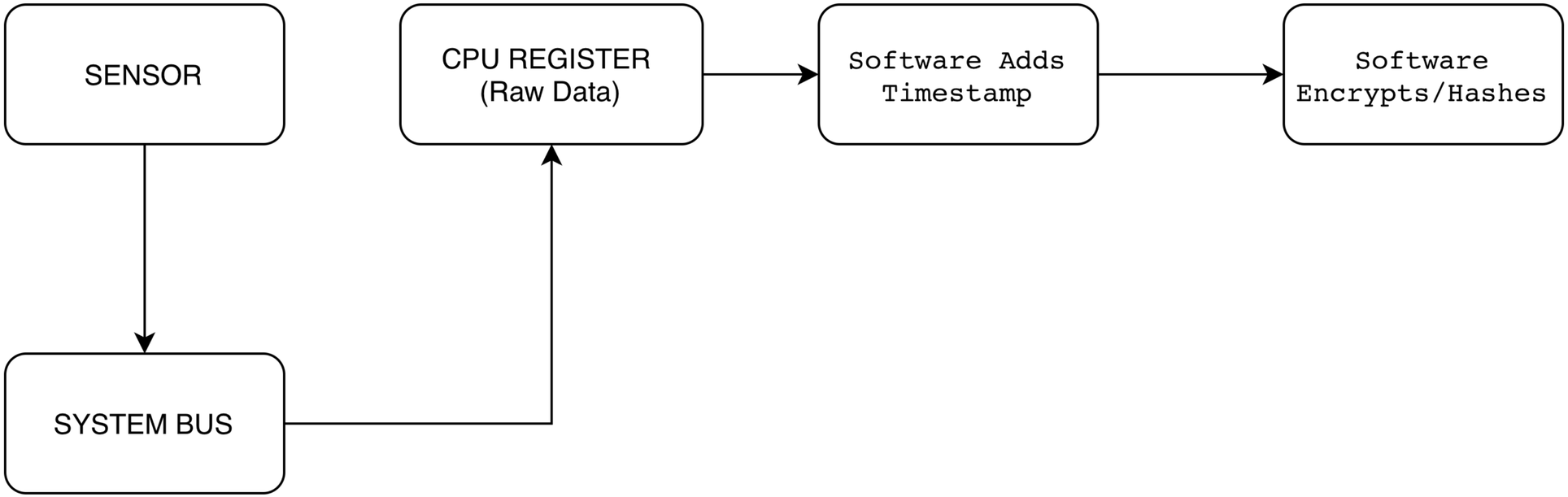
- **Industry Shift to Open-Source Security:** Our architecture mirrors the philosophy of **OpenTitan** (by Google & lowRISC), the world's first open-source silicon Root of Trust, proving the industry demand for transparent hardware security.
- **Academic Validation: *IEEE Transactions (2024)* : "RISC-V Custom Instructions of Elementary Functions for IoT Endpoint Devices"** validates that modifying the `custom-0` opcode space is the most power-efficient way to secure Edge IoT.
- ***MDPI (2024):* "A Trusted Execution Environment RISC-V SoC"** supports our claim that isolating cryptographic processes in the datapath prevents OS-level software spoofing.

# Working of Encryption Model

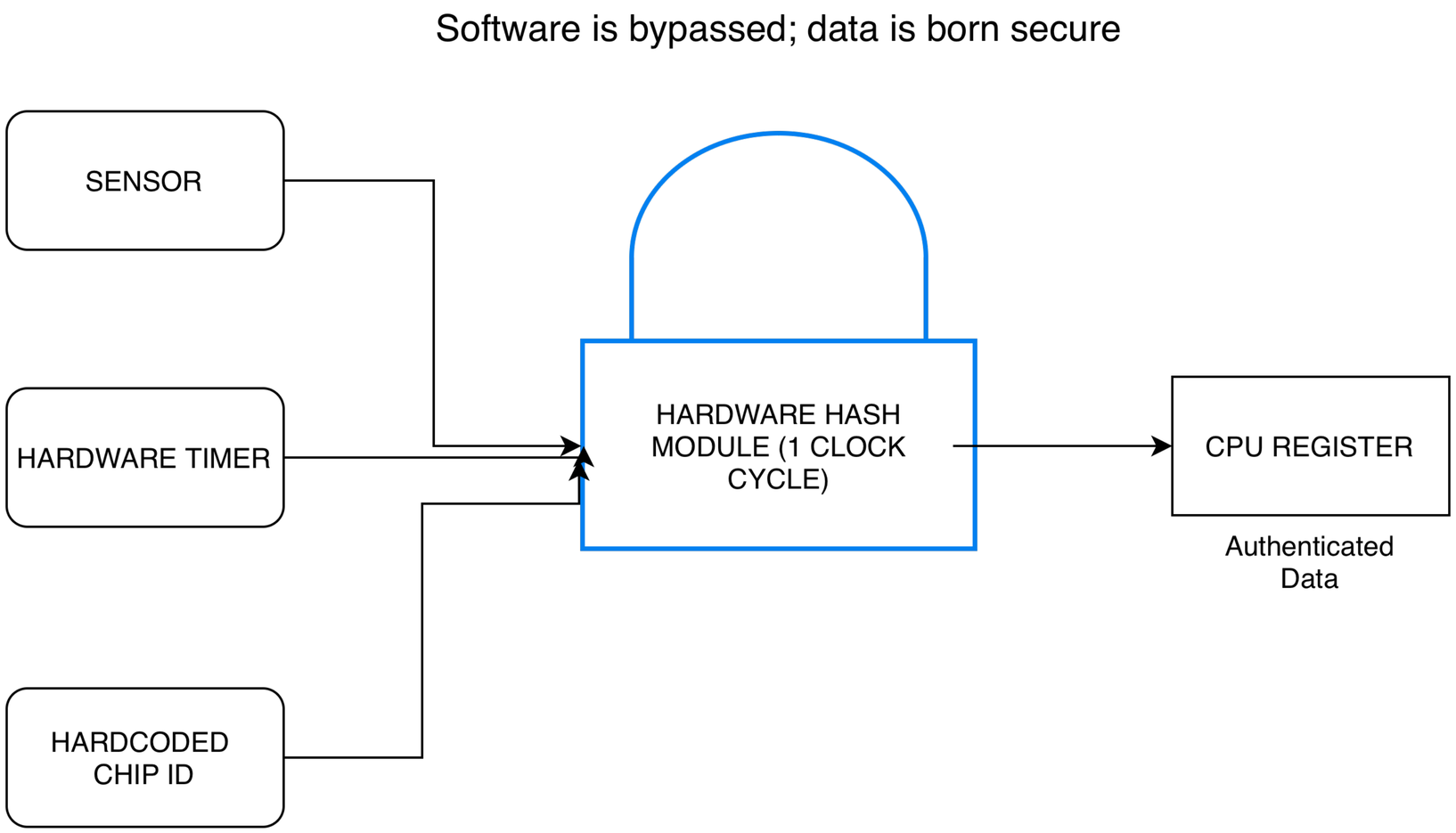
# STANDARD WAY OF DATA ENCRYPTION



**Vulnerability :** Software Interception



# HARDWARE ACCELERATED DATA ENCRYPTION




**Hardware Action:** The processor grabs the sensor data, current timer, and a unique Chip ID, hashing them all simultaneously at the hardware level before software even sees it.

# System Architecture Overview

## Custom CPU Core

**Base Core: Open-source Ibex RV32I soft-core processor deployed on a Spartan-7 FPGA**

Executes the custom **LDSENS** (Load Sensor Secure) operation and a custom hardware logic block to handle encryption of data before storing it in a register.

# LDSENS Microarchitecture

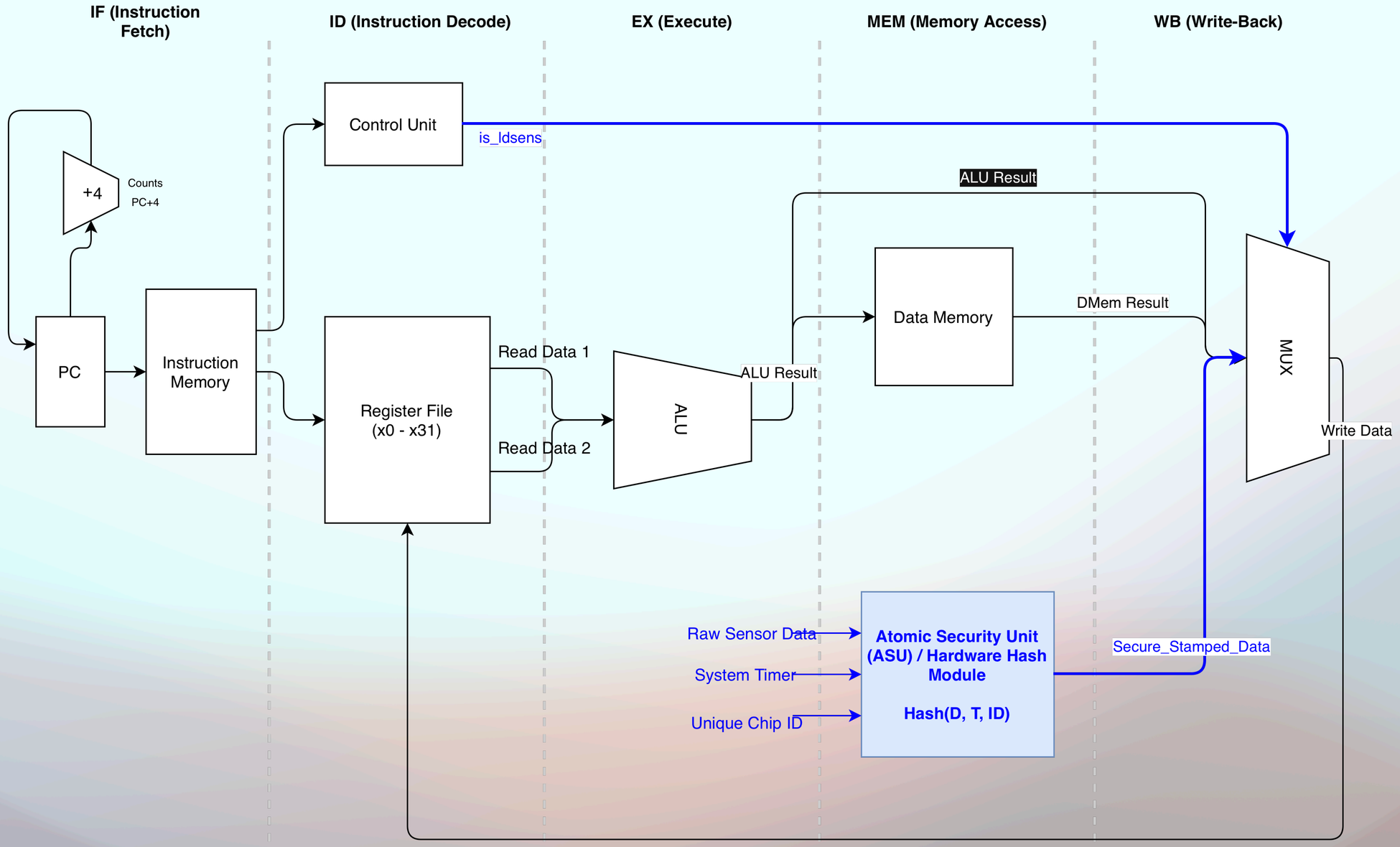
**Instruction Decode :** The Control Unit recognises the unique **LDSENS** opcode.

**Data-Path Integration :**

- Input 1 : Raw Sensor Data Bus
- Input 2 : Internal Hardware Timer/Counter
- Input 3 : Hardcoded Unique Chip ID Register

**Execution:** These three inputs bypass the standard register write-back and feed directly into a custom Cryptographic Hash Module.

# DATA-PATH BLOCK DIAGRAM



# Cryptographic Hash Module

## Hardware Accelerated Hash Module

**Design:** Combinational logic designed to execute in a single clock cycle (or a strictly controlled pipeline stall if using a multi-cycle hash). It will be a combinational circuit with a lightweight hash encoding system for single clock cycle implementation (XOR Shift Network).

**Function:** Mathematically binds the data, time, and identity into an immutable packet.

**Output:** The hashed, authenticated data packet is then written to the destination register for the software to safely use.

# Competitive Advantage: Why LDSENS?

| | |
|---|---|
| Traditional Software | • Drawback : High latency, drains IoT battery, leaves data exposed in registers. |
| Traditional Bus Based Coprocessors | • Drawback : Requires sending plain text over the system bus, high silicon area cost. |
| LDSENS Atomic Instruction | • The uniqueness : “Zero Cycle Vulnerability”. Data never exists in plaintext outside the ALU.<br>• The uniqueness : “Ultra Low Area”. By avoiding a separate coprocessor and utilising the existing CPU data path, it fits on low cost edge devices. |

# Development and Verification

## ASIC FLOW

The CPU design and verification will utilize software-based ASIC simulation methods before physical deployment to the FPGA

**RTL Design:** SystemVerilog implementation of the modified datapath and Control Unit.

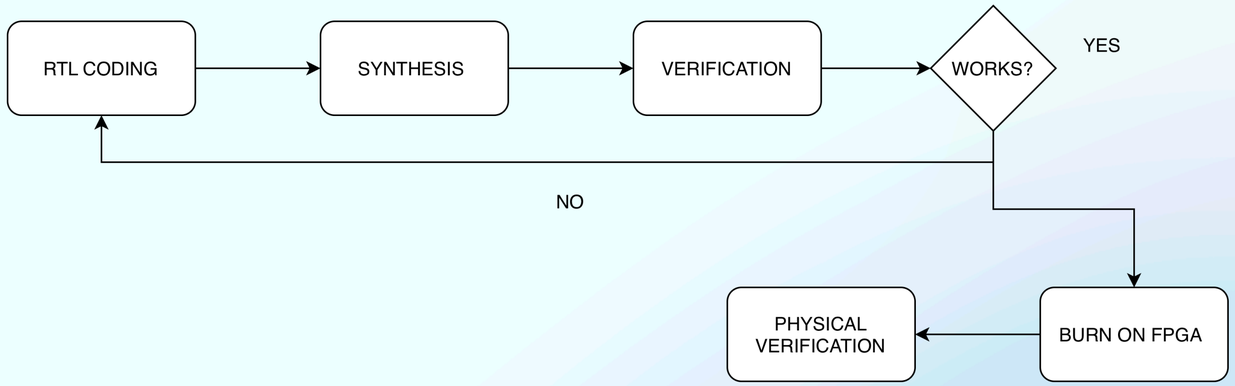
**Spoofing Test:** Proving that any software attempt to inject fake data fails the hardware authentication check.

**The Toolchain :**

- **RTL Design :** SystemVerilog/Vivado
- **Simulation/Synthesis :** Verilator, GTKWave and Yosys
- **Cross-Compilation :** RISC-V GNU Toolchain

# DESIGN FLOW

- The RTL coding will replicate a simulation model of the hardware for testing and verification.
- If the net-list is verified, the code is burned onto a physical FPGA board for hardware testing.
- The attack can be simulated using an Arduino board which tries to alter the data memory by spoofing fake data.



# Industry Relevance and Application

## Skill Development

**Hardware Security:** Directly addresses pressing needs in defense, banking, and critical infrastructure networks.

**VLSI & Architecture Focus:** Demonstrates practical mastery of datapath manipulation, control unit design, and RTL implementation.

**Performance:** Offloading this security to an atomic hardware instruction drastically reduces CPU cycles compared to software encryption.

# Conclusion

## FUTURE SCOPE

The hardware acceleration for encryption will prevent remote attacks on IoT based devices. **But there arises a question.**

**What if someone alters incoming data from sensor physically?**

While our LDSENS instruction successfully prevents software-level spoofing within the CPU, it does not protect against physical Man-in-the-Middle attacks on the external sensor bus. As a future expansion, we propose integrating an asymmetric signature verification module into our hardware pipeline. This would allow the CPU to authenticate encrypted data directly from the source sensor, providing true end-to-end zero-trust architecture."

# References


1. IEEE Transactions on Computers (2024): Y. Chen, X. Wang, S. Song, L. Feng, and Z. Wang, *"RISC-V Custom Instructions of Elementary Functions for IoT Endpoint Devices,"* IEEE, vol. 73, pp. 523-535. DOI: 10.1109/TC.2023.3338829.
2. MDPI Electronics (2024): B. Kieu-Do-Nguyen, K. Nguyen, et al., *"A Trusted Execution Environment RISC-V System-on-Chip Compatible with Transport Layer Security 1.3,"* MDPI, vol. 13, no. 13. DOI: 10.3390/electronics13132508.
3. OpenTitan Project: Google & lowRISC, *"OpenTitan: Open Source Silicon Root of Trust,"* [Online]. Available: https://opentitan.org
4. Technology Innovation Institute (2023): *"Zero Trust Secure RISC-V System,"* Secure Systems Research Centre Whitepaper.